## Unit – 1

### 1 . What is Node.js, and how does it differ from traditional web server environments?

1. Node.js is open-source, meaning it's free to use and its code can be modified by anyone.
2. It is cross-platform, so it runs on different operating systems like Windows, macOS, and Linux.
3. Node.js lets developers run JavaScript outside the browser, mainly on the server side.
4. It allows building server-side applications using JavaScript, which was traditionally used only in browsers.
5. Node.js is built on Google Chrome's V8 engine, which converts JavaScript into fast machine code.
6. It uses an event-driven architecture, meaning it handles multiple tasks without waiting for one to finish.
7. The non-blocking I/O model makes Node.js handle many connections at once, improving performance.
8. It is ideal for scalable and real-time applications like chat apps, online games, or streaming services.

### differ from traditional web server

**Language Usage**:
Traditional servers like Apache or PHP-based systems don't use JavaScript for backend,

while Node.js uses JavaScript for both frontend and backend.

**Blocking vs Non-blocking**:
Traditional environments handle requests in a blocking way (one after another),

while Node.js uses a non-blocking model — it handles many requests at once using callbacks.

**Threading Model**:
Node.js uses a single-threaded event loop to manage multiple clients efficiently.
Traditional servers often use multi-threading, creating a new thread for each request.

**Speed and Performance**:
Thanks to the V8 engine and non-blocking model, Node.js is usually faster and more scalable for real-time applications.

**Use Case**:
Node.js is ideal for real-time apps like chat apps, games, or live data apps.
Traditional servers are often used for content-heavy websites or apps needing complex logic processing.

## 2 . List and explain at least three key features of Node.js

1. **Event-Driven and Non-Blocking I/O Model**
   Node.js uses an event loop to handle multiple tasks at the same time. It processes I/O operations like file or database access without blocking other tasks.

2. **Single-Threaded Architecture**
   Node.js handles many connections using a single thread with event handling. This saves memory and keeps the server responsive even under heavy load.

3. **Fast Execution (V8 JavaScript Engine)**
   Built on Google's V8 engine, Node.js runs JavaScript as machine code. This improves speed and makes it suitable for high-performance apps.

4. **Cross-Platform**
   Node.js works on Windows, macOS, and Linux without major changes. It ensures easy deployment and portability across different systems.

5. **NPM (Node Package Manager)**
   NPM provides thousands of modules to add features like login or database access. It simplifies development and manages project dependencies easily.

6. **Scalability**
   Node.js can scale both vertically and horizontally to handle large traffic. It supports multiple users and devices efficiently with fewer resources.

7. **Real-Time Capabilities**
   Node.js is ideal for apps like chats and games that need instant updates. It uses WebSockets to enable two-way communication between client and server.

8. **Lightweight and Efficient**
   Node.js uses minimal CPU and memory, thanks to its event-driven model. It's perfect for building fast, scalable, and resource-efficient apps.

## 3 . How does the traditional web server model work? Compare it with the Node.js process model.

In the traditional web server model, servers follow a synchronous, request response cycle where each request from a client is handled one at a time.

Servers like Apache or IIS (Internet Information Services) are typically used in this model.

**Characteristics**

**Request-Response Cycle** - Each client request is handled one by one. The server processes the request fully and then sends back a response.

**Multi-threaded Model** - Every new request creates a new thread. This uses more memory and system resources, especially with many users.

**Blocking Nature** - If one request takes time (e.g., reading a file), it can block others. This causes delays and slows down overall performance.

**Disadvantages**

**High Memory Usage** - Creating a new thread for every request increases memory usage. It becomes inefficient during high traffic.

**Poor Scalability** - Handling many users is difficult due to blocking and heavy resource use. It doesn't scale well for large applications.

**Latency** - Requests are processed one by one, so users may experience delays. This lowers performance under load.

### Compare it with the Node.js process model.

**Thread Handling**
– Traditional servers create a new thread for each request, using more memory.
– Node.js uses a single thread with an event loop to handle multiple requests.

**I/O Operations**
– Traditional servers use blocking I/O, waiting for one task to finish before starting another.
– Node.js uses non-blocking I/O, allowing it to continue processing without delay.

**Concurrency**
– Traditional servers handle limited concurrent users due to thread limits.
– Node.js handles thousands of users at once with less system load.

**Performance**
– Traditional models may slow down under heavy traffic due to too many threads.
– Node.js stays fast and responsive due to its efficient event-driven model.

**Scalability**
– Traditional servers are harder to scale because they depend on hardware threads.
– Node.js is highly scalable both vertically and horizontally.

**Use Case**
– Traditional servers are suitable for heavy CPU-based tasks.
– Node.js is ideal for real-time apps like chats, games, and streaming platforms

### 4 . What is asynchronous programming in Node.js, and why is it important?

Asynchronous programming is a fundamental part of Node.js.

It allows the server to handle multiple tasks simultaneously without waiting for a task to complete before starting the next one.

**Callbacks**: Functions that are passed as arguments to be executed once an asynchronous operation is completed.

They are often used in Node.js to handle results of operations like reading from a file or querying a database.

```
fs.readFile('file.txt', 'utf8', function(err, data) {
  if (err) throw err;
  console.log(data);
});
```

**Promises**: A promise represents a value that is returned from an asynchronous operation.

It allows cleaner chaining of multiple asynchronous calls and better error handling.

```
let promise = new Promise((resolve, reject) => {
  let data = fetchDataFromDatabase();
  if (data) resolve(data);
  else reject('Error');
});
```

**Async/Await**: Introduced in ES2017, async/await syntax provides a more readable way to work with asynchronous code.

It makes asynchronous code look and behave more like synchronous code.

```
async function fetchData() {
  try {
    let response = await fetch('http://example.com');
    let data = await response.json(); // or response.text() if not JSON
    console.log(data);
  } catch (error) {
    console.error(error);
  }
}
```

**Why is it Important**

**Improves Performance**: - It lets Node.js handle **multiple requests at the same time** without waiting for one task to finish.

**Non-Blocking Execution**: - Long operations (like file I/O) don't stop the rest of the code from running, keeping the app fast and responsive.

**Efficient Resource Use**: - It reduces the need for multiple threads, using **less memory and CPU**.

**Ideal for Real-Time Apps**: - It's perfect for apps like **chat, games, and live data feeds**, which need quick responses.

**Scalability**: - Node.js can handle **thousands of concurrent users** efficiently due to async handling.

## 5 . What are the steps involved in setting up a Node.js development environment?

1. **Install Node.js and npm**
   Download and install Node.js from the official website (https://nodejs.org).
   npm (Node Package Manager) is included automatically with Node.js.
2. **Verify Installation**
   Open your terminal or command prompt and type:
   node -v and npm -v to check if Node.js and npm are installed correctly.
3. **Choose a Code Editor**
   Install a code editor like **Visual Studio Code (VS Code)** for writing and managing code.
   It supports extensions and debugging for Node.js.

4. **Initialize a Project**
   Create a new folder and open it in the terminal.
   Run npm init or npm init -y to create a package.json file that manages your project info and dependencies.

5. **Create Main JavaScript File**
   Create a file like app.js or index.js.
   This is where your Node.js application code will go.

6. **Install Required Packages**
   Use npm to install packages like express, mongoose, etc.
   For example: npm install express to add Express.js to your project.

7. **Run Your Application**
   In the terminal, run your app using: node app.js.
   You can also use tools like **nodemon** (npm install -g nodemon) for auto-reloading.

## 6 . Describe the advantages of using Node.js for backend development.

1. **Fast Performance**
   Node.js runs on Google's V8 engine, which compiles JavaScript to machine code. This makes the backend code execute very quickly. It helps in building high-performance applications.

2. **Non-Blocking I/O**
   Node.js uses an event-driven, asynchronous model to handle tasks. It doesn't wait for one task to finish before starting another. This allows it to handle many requests at once.

3. **Single Programming Language**
   JavaScript is used for both frontend and backend in Node.js. This reduces the need to switch languages in full-stack development. It makes coding, debugging, and collaboration much easier.

4. **Scalability**
   Node.js handles thousands of connections using fewer resources. It supports vertical and horizontal scaling for growing apps. This makes it great for large-scale, real-time applications.

5. **Large Ecosystem (npm)**
   Node.js has npm, the world's largest package manager. It offers thousands of free modules to add functionality quickly. This saves time and effort in development

6. **Real-Time Capabilities**
   Node.js supports WebSockets and real-time communication. It's ideal for apps like chat, live streaming, or online gaming. Real-time updates are fast and smooth.

7. **Cross-Platform Support**
   Node.js runs on Windows, macOS, and Linux without changes.You can develop and deploy apps easily across systems. This improves flexibility in development and deployment.

**7 . Explain the concept of the event-driven architecture in Node.js.**

**Definition**
Event-driven architecture means that the flow of the program is controlled by events.
Instead of running code in order, Node.js **waits for events** (like requests or user actions) and responds when they happen.

**How It Works**
Node.js uses an **event loop** to constantly listen for and handle events.
When an event occurs (like a file is read or a request comes in), the corresponding **callback function** is triggered.

**Single-Threaded, Non-Blocking**
Even though Node.js is single-threaded, it can handle **many tasks at once** using events.
It doesn't block or wait; instead, it continues working while the system handles I/O operations in the background.

**Example Scenario**
If a file is being read, Node.js doesn't wait for the file reading to complete.
It moves on and comes back to execute the callback once the file is ready — that's the event-based flow.

**Benefits**
This model is **fast, memory-efficient**, and perfect for apps with lots of I/O tasks.
It can handle **thousands of simultaneous users** with minimal resources.

**Use Cases**
Event-driven architecture is ideal for **real-time apps** like chat apps, live notifications, games, or streaming services.
It ensures **quick response times and smooth user experience**.

## 8 . List and describe at least three types of applications that can be developed using Node.js.

1. **Real-Time Applications**
   Node.js is ideal for apps needing instant updates like chats and games. It supports real-time, two-way communication using tools like **WebSockets** or **Socket.io**.
   **Example**: Chat apps and collaboration tools like Zoom or WhatsApp.

2. **Web Servers and APIs**
   Node.js is commonly used to build **RESTful APIs** and backend servers. Its non-blocking I/O handles multiple requests efficiently.
   **Example**: Creating APIs for e-commerce websites using **Express.js**

3. **Streaming Applications**
   Node.js handles streaming large data like video/audio in real time. It processes data in chunks instead of loading it all at once.
   **Example**: Streaming services like **Netflix** or **Spotify**.

4. **Serverless Applications**
   In serverless environments, Node.js runs lightweight functions on demand. No need to manage a server — it responds to events directly.
   **Example**: Using Node.js in **AWS Lambda** for cloud-based functions.

5. **Single Page Applications (SPAs)**
   Node.js works well with SPAs that need dynamic, real-time content. It powers backend logic and handles AJAX requests efficiently.
   **Example**: Dashboards showing live stock prices or news updates.

6. **Internet of Things (IoT) Applications**
   Node.js supports many simultaneous connections with low resource use. It's great for managing smart devices in real-time.
   **Example**: Backend for **smart home systems** like lights or sensors.

7. **Microservices Architecture**
   Its fast, lightweight nature makes Node.js ideal for microservices. Each service can be built and scaled independently.
   **Example**: Breaking an e-commerce app into services like login, orders, and cart.

## Unit – 2

### 1 . What is NPM (Node Package Manager)? Explain its importance in Node.js development.

NPM stands for Node Package Manager used to manage packages in Node.js projects.

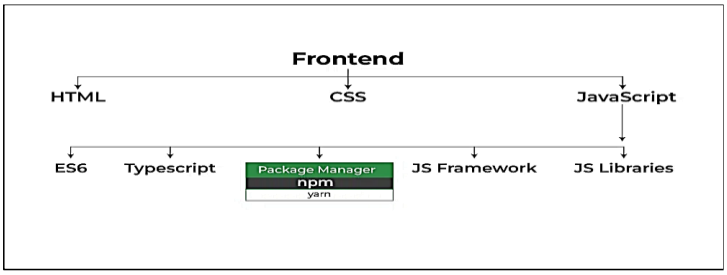
It is the default package manager that comes pre-installed with Node.js.

NPM allows installing, updating, and removing packages for easy module management.

It uses package.json to store and manage project dependencies.

Packages can be installed locally for projects or globally for system-wide use.

NPM provides access to a vast registry of third-party open-source library




### importance in Node.js development

1. **Access to Thousands of Packages**
   NPM provides access to the world's largest library of open-source packages, which helps developers add features like login, database access, etc.

2. **Faster Development**
   Instead of writing everything from scratch, developers can reuse existing modules, saving time and effort.
3. **Easy Dependency Management**
   NPM keeps track of all project dependencies in a package.json file, making project setup and sharing much easier.
4. **Version Control**
   NPM allows developers to install specific versions of a package, which ensures consistency across teams and environments.
5. **Automation Tools**
   NPM scripts can automate tasks like starting the server, running tests, or building the project, improving workflow efficiency.
6. **Community Support**
   Since NPM is widely used, it has a huge developer community that keeps contributing useful and updated packages.

## 2 . Describe the steps to install a module using NPM. Provide an example command

NPM allows developers to install packages or modules into their Node.js project or globally to be used across multiple projects.

### Installing Modules Locally:

When you install a module locally, it is installed in the node_modules folder in the project's root directory. This is the most common way of installing dependencies.

Local modules are only available within your project directory. They are not accessible globally across other projects.

Installed modules are recorded in package-lock.json. This file locks the exact versions for consistency.

- **Syntax**:

```
npm install name                <package->
```

- **Example**: Install express locally for a project:

```
npm install express
```

This will add express to the node_modules folder and update the **package.json** file with express as a dependency.

### Installing Modules Globally:

Global installations are useful when you need a package to be accessible system wide, regardless of the project you're working on. This is common for command line tools like eslint, nodemon, etc.

Installed global modules can be run from any terminal location. They are added to your system's PATH environment variable.

Use **npm uninstall -g <package-name>** to remove a global module. It deletes the module from the global storage and PATH.

**Syntax**:

```
npm install -g <package-name>
```

**Example:**

npm install -g nodemon

### 3 . What is the role of the package.json file in a Node.js project?

package.json is a central file in every Node.js project, serving as the metadata for the project.

It contains information about the project and its dependencies, scripts, versioning, and configuration settings.

When you install a module with NPM, it automatically updates the package.json file with the installed package information.

**Track dependencies** – Automatically stores names and versions of libraries your project uses.

**Define scripts** – Lets you create reusable command-line tasks under the scripts section.

**Versioning** – Tracks the version of your app and its dependencies for better control.

Run npm init to start creating a package.json file interactively. It asks for project details like name, version, and description.

You'll also be prompted for entry point, test command, and license. These help define the structure and purpose of the project.

To skip all prompts, use npm init -y for default values. It creates a basic package.json instantly with pre-filled data.

The generated file includes sections like name, version, and scripts. You can edit it anytime to update project configuration.

package.json is essential for managing dependencies and scripts. It makes the project portable, shareable, and easier to maintain.

### 4 . How can you uninstall a module using NPM? Write the command and explain the process.

To uninstall a module, you can use the npm uninstall command, followed by the package name.

This will remove the module from both node_modules and the package.json file.

**Updates the package.json file** by removing the module from the dependencies list.

**Global modules can be uninstalled**, which removes them from the system-wide environment.

After uninstallation, the module's code is no longer accessible in your project or globally.

You can verify uninstallation by checking package.json or using commands to list installed modules.

**Syntax**

```
npm uninstall <package-name>
```

**Example**

**npm uninstall express**

This command removes the express package from your project and updates the package.json and package-lock.json files accordingly.

**Uninstall globally**

```
npm uninstall -g <package-name>
```

**Example**

**npm uninstall -g express**

## 5 . How can you update an installed module using NPM? Provide the command and explain its usage.

**Basic command:** npm update <package-name> updates a specific package to the latest version allowed by package.json.

**Global update:** Use npm update -g <package-name> to update a globally installed module.

**Bulk update:** Running npm update without a package name updates all dependencies listed in package.json.

**Version rules matter:** It only updates to versions that match the semver rules (^, ~) in package.json.

**Doesn't change package.json version manually:** The version in package.json remains the same unless the update is beyond the defined range.

**Check outdated packages:** Use npm outdated to see current, wanted, and latest versions of all dependencies.

**Force install specific version:** If you want the latest version regardless of package.json, use npm install <package-name>@latest.

**Lock file update:** Updates are reflected in package-lock.json to ensure consistency across environments.

**Useful for patching bugs or security:** Keeps packages up-to-date to fix known issues or vulnerabilities.

**Compatible with semantic versioning:** Ensures safe and controlled updates based on version constraints.

## 6 . Differentiate between Global Installation and Local Installation of a module. When should each be used?

| Feature | Local Installation | Global Installation |
|---|---|---|
| Scope | Modules are installed only within the current project. | Modules are installed globally, available across all projects. |
| Directory | Installed in the project's `node_modules` folder. | Installed in a global directory (e.g., `/usr/local/lib`, or system path). |
| Usage | For adding project-specific dependencies. | For tools/utilities used from CLI or across projects. |
| package.json | Automatically updates `package.json` and `package-lock.json`. | Not tracked in `package.json`. |
| Example | `npm install express` | `npm install -g nodemon` |
| Availability | Available only for that project. | Available system-wide from any terminal. |

**When to Use Local and Global Installation**

Use **local installation** when the module is required by the project code (e.g., express, mongoose).

Use **global installation** for CLI tools used in multiple projects (e.g., nodemon, eslint).

Install **locally** if you want the package tracked in package.json.

Install **globally** if the tool is meant to run from the command line anywhere.

Use **local** for dependencies that are specific to one project only.

Use **global** for packages you reuse often across many projects.

## 7 . Explain at least five key attributes of the package.json file and their significance.

**name** – Defines the project's name used for identification and publishing to npm.
It should be unique and lowercase without spaces.

**Example**: "name": "my-app"

**version** – Indicates the current version using semantic versioning (major.minor.patch).
It helps track updates and manage compatibility.

**Example**: "version": "1.0.0"

**description** – Provides a brief overview of what the project does.
It's shown in the npm registry and helps users understand the purpose.

**Example**: "description": "A simple web app"

**main** – Specifies the entry point file that runs when the module is imported.
Typically points to the main JavaScript file.

**Example**: "main": "index.js"

**scripts** – Defines custom commands run using npm run <script-name>.
Useful for automation like starting servers or running tests.

**Example**:

json

CopyEdit

"scripts": {

"start": "node index.js",

"test": "mocha"

}

**repository** – Provides the URL to the source code repository.
Helps others find, clone, and contribute to your project.

**Example**: "repository": "https://github.com/username/project.git"

**author** – Names the person or organization who created the project.
Gives credit and shows ownership or responsibility.

**Example**: "author": "Jane Doe"

**license** – Declares how others can use or distribute your project.
Ensures legal usage terms are clear to everyone.

**Example**: "license": "MIT"

## 8 . What is the purpose of the NPM registry, and how does it help developers?

### Purpose

Hosts a vast collection of open-source Node.js packages for public use. It serves as the official hub for reusable JavaScript modules.

Acts as a centralized storage for package metadata and code. Developers can search and access thousands of modules easily.

Allows developers to publish and share their custom packages globally. This promotes reuse and contribution within the community.

Provides version control to track changes and updates in packages. Ensures projects stay compatible and up to date.

Supports collaboration by letting multiple developers contribute to a package. Enhances teamwork and open-source contributions.

Maintains package availability through a reliable, always-online platform. Packages are accessible anytime during development.

### how does it help developers

Saves time by allowing reuse of pre-built and tested modules. No need to reinvent common functionality.

Simplifies installation and management of project dependencies. Just one command installs everything needed.

Makes it easy to track, update, or remove libraries as needed. Helps maintain a clean and stable codebase.

Offers a trusted source of community-vetted solutions. Developers benefit from stable and popular tools.

Lets developers publish and share their own packages. Builds reputation and encourages knowledge sharing.

Standardizes project setup using known tools and patterns. Speeds up onboarding for new team members.

## Unit – 3

### 1 . How can we Create, Read, and Remove a Directory in Node js.

Node.js's fs module also provides methods to create, read, and remove directories.

**Creating a Directory**

```
fs.mkdir('newdir', (err) => {
  if (err) throw err;
  console.log('Directory created');
});
```

**Explanation:**
Creates a new folder named newdir in the current directory. If an error occurs (like if the folder already exists), it throws an error. If successful, it logs "Directory created" to the console.

**Reading a Directory**

```
fs.readdir('newdir', (err, files) => {
  if (err) throw err;
  console.log(files);
});
```

**Explanation:**
Reads the contents of the newdir directory. Returns an array of file and folder names inside it.
If successful, logs the list; otherwise, throws an error.

### Removing a Directory

```
fs.rmdir('newdir', (err) => {
  if (err) throw err;
  console.log('Directory removed');
});
```

**Explanation:**
Deletes the newdir directory from the file system. It must be empty; otherwise, an error is thrown. If successful, logs "Directory removed" to the console.

## 2 . Explain the difference between synchronous and asynchronous file system operations in Node.js. Why might you choose one over the other in different scenarios?

| Aspect | Synchronous (Blocking) | Asynchronous (Non-blocking) |
|---|---|---|
| **Execution** | Code waits for the file operation to finish before moving on. | Code continues running without waiting for the operation to finish. |
| **Thread Blocking** | Blocks the main thread during execution, halting other tasks. | Frees the event loop, allowing other tasks to continue in parallel. |
| **Performance** | Slower in high-load situations due to sequential execution. | Faster and more efficient for I/O-heavy operations. |
| **Scalability** | Not suitable for handling many operations at once. | Scales well with multiple users or concurrent requests. |
| **Error Handling** | Uses `try/catch`, which is straightforward for managing errors. | Errors handled via callbacks, promises, or async/await, depending on the pattern used. |
| **Code Simplicity** | Code is simpler and easier to read in short scripts. | Can become complex with nested callbacks, though async/await improves readability. |
| **Use Case** | Best for startup scripts, CLI tools, or setup tasks. | Ideal for servers, APIs, or apps that handle multiple users or I/O. |
| **User Experience** | May cause UI or app to freeze during operations. | Keeps applications responsive by running tasks in the background. |
| **Resource Usage** | Inefficient use of CPU and memory under load. | Optimized resource usage thanks to non-blocking architecture. |
| **Syntax** | Example: `fs.readFileSync('file.txt')` for blocking read. | Example: `fs.readFile('file.txt', callback)` for non-blocking read. |

### choose one over the other in different scenarios

1. Use asynchronous in web servers to handle multiple requests efficiently.
2. Choose synchronous in CLI tools or scripts where execution order matters.
3. Use asynchronous to prevent blocking the main thread in large apps.
4. Prefer synchronous for small, quick tasks in simple workflows.
5. Asynchronous is better for performance and scalability.

### Example

### Writing to a File (Synchronously)

```
fs.writeFileSync('example.txt', 'Hello, Node.js!', 'utf8');
```

**Writing to a File (Asynchronously)**

```
fs.writeFile('example.txt', 'Hello, Node.js!', 'utf8', (err) => {
  if (err) throw err;
  console.log('File has been written');
});
```

**3 . Create a program that takes user input and writes it to a new text file. If the file already exists, append the new data to the end of the file.**

Copy Edit

```
const fs = require('fs');
const readline = require('readline');

// Setup input interface
const rl = readline.createInterface({
  input: process.stdin,
  output: process.stdout
});

// Prompt user for input
rl.question('Enter the text you want to write to the file: ', (userInput) => {
  const fileName = 'userInput.txt';
  const dataToWrite = userInput + '\n';

  // Append data to the file (or create if it doesn't exist)
  fs.appendFile(fileName, dataToWrite, 'utf8', (err) => {
    if (err) {
      console.error('Error writing to file:', err);
    } else {
      console.log(`Data saved to ${fileName}`);
    }
    rl.close();
  });
});
```

**4 . Write syntax of Appending to a File, Renaming a File, Truncating a File.**

**Appending to a File**

```
fs.appendFile('example.txt', 'Appended text', 'utf8', (err) => {
  if (err) throw err;
  console.log('Data appended');
});
```

**Explanation:**
Adds 'Appended text' to the end of example.txt. If the file doesn't exist, it creates one automatically. Executes asynchronously and logs once done.

### Renaming a File

```
fs.rename('oldname.txt', 'newname.txt', (err) => {
  if (err) throw err;
  console.log('File renamed');
});
```

**Explanation:**
Changes the file name from oldname.txt to newname.txt. Does not change content, just renames the file.  Throws error if source file doesn't exist.

### Truncating a File

```
fs.truncate('example.txt', 10, (err) => {
  if (err) throw err;
  console.log('File truncated');
});
```

**Explanation:**
Reduces example.txt content to the first 10 bytes. Deletes everything beyond the 10th character. Used to shorten or reset file content partially.

### 5 . Write a program using Node.js that sends an HTTP request to an API

```
const express = require('express');
const app = express();
const PORT = 3000;

app.get('/hello', (req, res) => {
  res.send('Hello from Node.js API!');
});

app.get('/info', (req, res) => {
  res.json({ name: 'Node API', version: '1.0.0' });
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

We use **Express** – a Node.js framework to create web servers easily.

Install it using: **npm install express**

**const app = express();**

Makes a new Express app to define routes and handle requests.

**app.get('/hello', (req, res) => {**

**res.send('Hello from Node.js API!');**

**});**

When someone visits /hello, it replies with simple text.

**app.listen(PORT, () => {**

**console.log(`Server running at http://localhost:${PORT}`);**

**});**

Starts the server and prints the running message.

## Unit – 4

### 1 . Define the concept of routing in Express.js. How do routes work in handling different HTTP methods (GET, POST, PUT, DELETE)?

A route in Express.js refers to the definition of how an application should respond to a client request for a specific endpoint, defined by a URI (Uniform Resource Identifier) and a specific HTTP method (GET, POST, PUT, DELETE, etc.).

Express uses functions like app.get(), app.post(), etc. to handle routes. Each method corresponds to a different type of HTTP request.

Routing keeps code organized and readable by separating logic for each endpoint. Each route function handles only its assigned task.

**app.METHOD(PATH, HANDLER);**

**METHOD** is the HTTP method (e.g., get, post, put, delete).

**PATH** is the URL or route (e.g., /users, /products/:id).

**HANDLER** is the callback function that gets executed when the route is matched.

**GET**

Used to retrieve or fetch data from the server.It's mostly for reading — no changes are made to data.
**Example**: GET /users returns a list of users.

**POST**

Used to send new data to the server to create something. Commonly used in forms and APIs to add entries.
**Example**: POST /users adds a new user to the database.

**PUT**

Used to update existing data or replace it completely.The entire object is usually sent for update.
**Example**: PUT /users/5 updates user with ID 5.

**DELETE**

Used to remove data from the server permanently.Once deleted, the data is no longer available.
**Example**: DELETE /users/5 removes the user with ID 5.

## 2 . What is the difference between route parameters and query strings in Express.js?

### route parameter

Route parameters are part of the URL and are used to capture values specified at certain positions in the URL. They are defined with a colon (:) in the route path.

Defined in the URL path using a colon, e.g., /user/:id.

Used for required data like IDs, names, or resource identifiers.

Accessed in Express using req.params.

Best for URLs that describe specific resources.

**Example**:

```
app.get('/user/:id', (req, res) => {
  res.send(`User ID: ${req.params.id}`);
});
```

id is a **dynamic value** in the URL. It can change based on the user's request.

**req.params** object holds all the route parameters.

### query strings

Query strings are appended to the URL using a question mark (?) and are typically used to filter or sort data.

Used for optional data like filters, sorting, or search terms.

Accessed in Express using req.query.

Ideal for passing flexible or multiple optional values.

Can be skipped or added freely in the URL.

**Example**:

```
app.get('/search', (req, res) => {
  const query = req.query.q;
  res.send(`Searching for: ${query}`);
});
```

**req.query** object contains all the query string parameters (e.g., /search?q=express).

It uses a **query string**, like /search?q=books.

### 3 . How do middleware functions work in Express, and how do they interact with the request response cycle?

Middleware functions in Express are functions that have access to the request (req), response (res), and the next middleware function in the request-response cycle.

Middleware functions can modify the request, response, or end the request-response cycle.

It can access the **request (req)** and **response (res)** objects to read, modify, or log data.

Calling next() inside middleware passes control to the **next middleware** or **route handler**.

Some middleware (like error handlers or response senders) can **end the request-response cycle** directly.

```
app.use((req, res, next) => {
  console.log('Request received at ' + Date.now());
  next(); // Passes control to the next middleware
});
```

**app.use()** sets up a **middleware function** that runs for every incoming request.

**next()** allows the request to continue to the next middleware or route handler.

This is useful for **logging, tracking, or debugging** request activity.

**Types of Middleware**

**Built-in Middleware:**

○ express.static(): Serves static files.

○ express.json(): Parses incoming JSON data.

**Custom Middleware:** Functions you create to handle specific tasks like logging, authentication, etc.

Error Handling Middleware: ● Error-handling middleware must be defined with four arguments: err, req, res, next.

```
app.use((err, req, res, next) => {
  res.status(500).send('Something went wrong!');
});
```

**4 . What are REST APIs and how does Express.js help in building them? Explain with an example.**

REST (Representational State Transfer) is an architectural style that is widely used to build web services. RESTful APIs rely on stateless communication and use standard HTTP methods.

**Basic HTTP Methods:**

**GET:** Retrieve data.

**POST:** Create new data.

**PUT:** Update existing data**.**

**DELETE:** Remove data.

A **REST API** allows clients to access and manipulate data using URLs and HTTP verbs.

**Express.js** is a fast, minimal Node.js framework ideal for building REST APIs.

Express provides methods like app.get(), app.post(), etc. to easily define RESTful routes.

You can structure endpoints to represent resources like /users, /products, etc.

Express allows adding **middleware** for tasks like authentication, logging, and parsing data.

Express can handle **JSON data**, making it perfect for modern APIs that exchange JSON.

**Example**

```
const express = require('express');
const app = express();
app.use(express.json()); // To read JSON data

let users = [{ id: 1, name: 'John' }];

app.get('/users', (req, res) => res.json(users));          // Read
app.post('/users', (req, res) => {                         // Create
  users.push(req.body);
  res.send('User added');
});

app.listen(3000, () => console.log('API running'));
```

## Unit – 5

### 1 . How would you handle errors and connection issues when connecting to an RDBMS in Node.js? Write a simple example of error handling when connecting to a MySQL database.

Always check for errors when connecting to the database using callbacks or try-catch. This helps catch login issues, wrong configs, or unreachable servers early.

Use **try...catch** when working with async/await to catch promise rejections. This prevents the app from crashing due to unhandled errors.

Log detailed error messages for developers, but never expose them to end-users. This keeps debugging easy while maintaining security.

Implement auto-reconnection logic if the connection is lost due to server timeout or crash. It ensures your app can recover without restarting.

Store credentials like DB user and password in environment variables. This avoids hardcoding sensitive data in your source code.

Validate SQL queries and handle common failures like syntax errors or missing tables. It prevents your app from breaking on bad inputs or database changes.

### Example

```
const mysql = require('mysql');

const connection = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: 'wrongpass',
  database: 'testdb'
});

connection.connect((err) => {
  if (err) {
    console.error('Connection failed: ', err.message);
    return;
  }
  console.log('Connected to MySQL!');
});
```

### 2 . What is MongoDB and how does it differ from a traditional RDBMS? Explain the key differences between them.

#### What is MongoDB

1. **MongoDB is a NoSQL database** that stores data in a flexible, JSON-like format called BSON.
2. It is document-based, meaning data is stored in documents inside collections, not tables.

3. MongoDB is schema-less, allowing you to store different structures in the same collection.
4. It is designed for high performance, scalability, and handling large volumes of unstructured data.
5. It supports powerful querying, indexing, aggregation, and replication features.

### key differences between them.

**Data Structure**:
MongoDB stores data in JSON-like documents with key-value pairs.
RDBMS stores data in rows and columns inside tables.

**Schema**:
MongoDB is schema-less, so documents in a collection can have different structures.
RDBMS requires a predefined schema with fixed column types and structure.

**Scalability**:
MongoDB supports horizontal scaling through sharding across multiple servers.
RDBMS usually relies on vertical scaling, which has hardware limitations.

**Joins**:
RDBMS supports complex joins between multiple tables using foreign keys.
MongoDB uses embedded documents and manual joins instead of built-in joins.

**Query Language**:
RDBMS uses SQL (Structured Query Language) to manage and query data.
MongoDB uses a JSON-style query syntax that matches the document structure.

**Transactions**:
RDBMS offers robust ACID transactions with strong consistency guarantees.
MongoDB supports transactions but is optimized for flexibility and speed.

**Use Case**:
MongoDB is ideal for unstructured or fast-changing data like logs, chats, etc.
RDBMS is better for structured data with clear relationships, like banking systems.

**3 . Using MongoDB and Mongoose, write a query to insert a new document into a collection (e.g., products) with fields like name, price, and category.**

```
const mongoose = require('mongoose');

// Connect to MongoDB
mongoose.connect('mongodb://localhost:27017/shopDB');

// Define schema and model
const Product = mongoose.model('Product', {
  name: String,
  price: Number,
  category: String
});

// Create and insert document
Product.create({
  name: 'Smartphone',
  price: 699,
  category: 'Electronics'
})
.then(() => console.log('Product inserted'))
.catch(err => console.error('Insert failed:', err));
```

**4 . Explain the DELETE command in SQL. Write an SQL query that deletes a user with id = 10 from a users table.**

**DELETE command in SQL.**

The DELETE command is used to remove one or more rows from a table.

It permanently removes data from the table unless a transaction is used.

The WHERE clause is essential to specify which rows to delete.

Without WHERE, it deletes **all rows** from the table (use with caution).

It only deletes data, not the table structure or schema.

It can be combined with conditions using operators (=, >, <, etc.).

It's useful for removing outdated, duplicate, or specific records.

**Example**

```
const mysql = require('mysql');

// Create database connection
const db = mysql.createConnection({
  host: 'localhost',
  user: 'root',
  password: '',
  database: 'mydb'
});

// Connect to database
db.connect((err) => {
  if (err) throw err;
  console.log('Connected to MySQL');

  // Delete query
  const sql = 'DELETE FROM users WHERE id = 10';
  db.query(sql, (err, result) => {
    if (err) throw err;
    console.log('User deleted:', result.affectedRows);
  });
});
```

## 5 . Explain the importance of using a WHERE clause when updating records in an RDBMS. What could happen if the WHERE clause is omitted?

**Importance of WHERE Clause in UPDATE**

It restricts updates to only those rows that meet specific conditions. This prevents unintended changes to the entire dataset.

Ensures that only the correct records are updated based on criteria. This improves data accuracy and reliability.

Protects valid data by avoiding accidental overwriting. This is essential for maintaining trust in your database operations.

Maintains database integrity, especially with related tables. Wrong updates can break key relationships and cause issues.

Improves performance by updating only necessary rows. This avoids full table scans and saves resources.

Gives developers clear control over which rows get updated. This adds precision to your SQL operations.

**WHERE clause is omitted**

All rows in the table will be updated with the new values. This can lead to major, unintended data loss.

Unique or meaningful values across rows may be overwritten. This causes irreversible changes unless backed up.

Business logic relying on that data could fail or behave incorrectly. Applications may show wrong outputs or crash.

Relationships with other tables might break due to changed keys. This can lead to orphaned or invalid foreign keys.

The update will require more time and processing power. Performance drops drastically for large tables.

It’s very difficult to detect and undo mass updates. You may spend hours debugging or restoring backups.